



Because Accessibility Report

WCAG 2.2 · LEVEL AA

Accessibility features of Because, a free browser-based editor for argument maps
Prepared for university IT and disability-services offices

July 2026 | app.philmaps.com | sc@simoncullen.org

CONFORMANCE SUMMARY

Conformance target: WCAG 2.2 Level AA

Because is designed and tested against the Web Content Accessibility Guidelines (WCAG) 2.2 at Level AA. Every part of the application interface is operable from the keyboard alone, exposes standard WAI-ARIA semantics for screen readers, meets AA contrast requirements, and conveys argument structure by shape and text in addition to color. Conformance is enforced by an automated regression gate — axe-core rule scans plus a scripted keyboard walkthrough, run in the WebKit browser engine — that must pass before any version is deployed. The current release passes all of its checks with zero WCAG violations reported.

AAWCAG 2.2 CONFORMANCE
TARGET**0**AXE-CORE VIOLATIONS, 5
UI STATES**47/47**

AUTOMATED CHECKS PASSING

100%COMMANDS KEYBOARD-
REACHABLE

About the application

Because (<https://app.philmaps.com>) is a free, browser-based editor for argument maps — diagrams that display the structure of an argument as a tree of claims, reasons, objections, and co-premises. It is used in university philosophy and critical-thinking courses. The application runs entirely in the browser: there is nothing to install, no account is required, and maps are stored in files the user keeps on their own computer or, optionally, in their own Google Drive. It renders the same .mup file format used by MindMup's argument-visualization tools, so existing course materials open without conversion.

Evaluation method

This report is a self-evaluation by the developer, dated July 2026; it is not a third-party audit. The evidence behind it is (a) automated scans with axe-core (Deque's accessibility engine) using the complete WCAG 2.0, 2.1, and 2.2 Level A and AA rulesets, run against five interface states — the welcome dialog, the map view in light mode and in dark mode, the keyboard-shortcuts panel, and the node-style editor; (b) a scripted, 47-step keyboard and screen-reader-semantics walkthrough covering focus order, dialog behavior, menu navigation, and editing commands; and (c) manual review. Automated testing runs in WebKit, the browser engine underlying Safari, which is the engine VoiceOver users on macOS and iOS depend on, and additionally asserts against the accessibility tree that Chrome computes — the representation Windows screen readers such as NVDA and JAWS consume — rather than only the markup as authored. The full test suite re-runs before every deployment, so the published version is always the version that passed.

SECTION 1

SCREEN READER SUPPORT

The interface is built from native HTML controls carrying standard WAI-ARIA roles, states, and properties — the vocabulary screen readers such as VoiceOver, JAWS, and NVDA are designed to consume.

- **Application menus follow the ARIA menubar pattern.** The top menu bar is a true menubar: menu titles are buttons with `aria-haspopup`, the whole bar is a single Tab stop navigated with the arrow keys (roving tabindex), and dropdown

entries are announced as menu items — including checkable ones, such as Dark mode, which expose `menuItemcheckbox` with a live checked state.

- **The map itself is exposed as an ARIA tree.** Although the map is a freeform visual canvas, assistive technology sees a structured tree: each claim is a `treeitem` whose depth (`aria-level`), expanded state (`aria-expanded`), and selection (`aria-selected`) are reported. Keyboard focus sits on the claim itself and moves with the arrow keys, so a screen reader announces each claim from native focus events — the mechanism with the broadest assistive-technology support — as the user moves through the argument. The decorative connector artwork is hidden from assistive technology because the relationships it draws are already conveyed by the tree structure.
- **Dialogs are real modal dialogs.** Every overlay — the welcome screen, information panels, the unsaved-changes prompt, the node-style editor — has `role="dialog"` with `aria-modal`, is labelled by its heading, traps Tab within itself while open, closes on Escape, and returns focus to the element that opened it.
- **Save status is announced automatically.** The save indicator is a `role="status"` live region, so screen readers report changes ("Saving...", "Saved") without the user having to look for them.
- **Toolbar buttons are labelled.** Every icon button carries an `aria-label`; the icons themselves are marked decorative.
- **Document basics are in place.** The page declares its language, uses real `<button>` elements throughout, and marks its chrome regions (menubar, toolbar, status) with the appropriate roles.

SECTION 2

KEYBOARD ACCESS

Every command in Because can be executed without a mouse.

- **Skip link.** The first Tab stop is a "skip to map" link, so keyboard users reach the content in one step (WCAG 2.4.1).
- **Complete command coverage.** All editing operations are in the menus, which are fully keyboard-operable. This includes operations whose mouse targets are small — the thin connector lines between claims — which have first-class menu equivalents (Edit connector label, Stronger connector, Weaker connector).
- **Efficient map editing.** With focus on the map, single keys do the frequent work: Enter adds a reason, Tab adds a co-premise, arrows move the selection, and so on. A reference panel (Help > Keyboard shortcuts) lists them all, and is itself an accessible dialog.
- **Shortcuts cannot fire by accident.** Single-key shortcuts are active only when focus is in the map (WCAG 2.1.4). Typing in a text field, a dialog, or the menus is never intercepted as a command.
- **Focus is always visible.** All chrome controls show a focus ring (WCAG 2.4.7). On the canvas, where the arrow keys move an ARIA selection rather than DOM focus, the selected claim is marked with a high-visibility 3-pixel outline whenever the map has keyboard focus.
- **No keyboard traps.** Escape closes any menu, dialog, or inline editor and returns focus to where it was; the automated suite verifies each of these paths.

SECTION 3

LOW VISION, COLOR, AND MOTION

Color is never the only cue

Argument maps traditionally distinguish supporting reasons (green) from objections (red). For users who cannot rely on color, Because encodes the same distinction in shape: an objection's bracket is drawn with square corners, while a reason's bracket is rounded. Structure is also carried redundantly in several other ways — the high-impact theme labels brackets with the words "because..." and "but...", implicit claims have dashed borders, claims are numbered, and focus and selection are shown with outlines and ARIA state rather than color alone.



Figure 1. The reason/objection distinction is carried by shape as well as color: reason brackets have rounded corners, objection brackets are square (WCAG 1.4.1, Use of Color).

Contrast

All text and controls in the application interface meet the AA thresholds — 4.5:1 for text, 3:1 for user-interface components — measured against the background each color actually sits on. Representative values:

ELEMENT	FOREGROUND	BACKGROUND	RATIO	REQUIREMENT
Links, buttons, accents	#16749f	#ffffff	5.21:1	4.5:1
Menu hover / open state	#16749f	#eef6fb	4.76:1	4.5:1
Save-status text (light mode)	#6e6e6e	#ffffff	5.10:1	4.5:1
Save-status text (dark mode)	#a2a9b0	#24272b	6.31:1	4.5:1
Primary button hover	#ffffff	#11577a	7.87:1	4.5:1
Focus outline	#16749f	#ffffff	5.21:1	3:1

Dark mode

A full dark theme (View > Dark mode) provides an alternative luminance scheme for light-sensitive users. It is a pure view preference: switching it never changes the map file, which saves byte-for-byte identically in either mode.

Zoom and reflow

The application chrome reflows without horizontal scrolling at narrow widths and high zoom levels; the automated suite verifies that every toolbar control remains visible and usable at a 640-pixel viewport. The map canvas itself is two-dimensional content of the kind WCAG's reflow criterion explicitly exempts (its spatial layout is its meaning); it provides its own zoom controls, keyboard-accessible from the toolbar and shortcuts.

Reduced motion

When the operating system signals a motion preference (`prefers-reduced-motion`), interface transitions and animations are disabled. This goes beyond the AA bar — the corresponding WCAG criterion (2.3.3) is Level AAA.

SECTION 4

WCAG 2.2 CONFORMANCE TABLE

Status terminology follows the ITI VPAT convention ("Supports", "Supports with exceptions"). The table lists the criteria most relevant to this application; the automated gate additionally covers the complete WCAG 2.0/2.1/2.2 Level A and AA axe-core rulesets.

SUCCESS CRITERION	LEVEL	STATUS	REMARKS
1.1.1 Non-text Content	A	Supports	Icon buttons carry text labels; decorative artwork is hidden from assistive technology.
1.3.1 Info and Relationships	A	Supports	Menubar, tree, dialog, and status semantics expose structure programmatically.
1.4.1 Use of Color	A	Supports	Shape (square vs. rounded brackets), text labels, dashed borders, and numbering supplement color.
1.4.3 Contrast (Minimum)	AA	Supports with exceptions	All application UI meets 4.5:1. Some colors inside user maps follow the historical MindMup theme (see Known Limitations).
1.4.4 Resize Text / 1.4.10 Reflow	AA	Supports	Chrome reflows at narrow widths; the map canvas is two-dimensional content (exempt) with its own zoom.
1.4.11 Non-text Contrast	AA	Supports	Toolbar icons and focus indicators meet 3:1.
2.1.1 Keyboard / 2.1.2 No Keyboard Trap	A	Supports	All commands keyboard-reachable; Escape exits every menu, dialog, and editor.
2.1.4 Character Key Shortcuts	A	Supports	Single-key shortcuts operate only while focus is in the map.
2.4.1 Bypass Blocks	A	Supports	A skip link is the first Tab stop.
2.4.3 Focus Order	A	Supports	Logical order; dialogs restore focus to their opener.
2.4.7 Focus Visible	AA	Supports	Focus rings on all chrome; a dedicated selection outline on the canvas.
2.5.8 Target Size (Minimum)	AA	Supports with exceptions	Connector lines are thin pointer targets; every connector action has a menu equivalent.
3.1.1 Language of Page	A	Supports	lang="en" declared.
3.2.1 On Focus / 3.2.2 On Input	A	Supports	Focus and input never trigger unexpected context changes.
4.1.2 Name, Role, Value	A	Supports	Verified by axe-core across five interface states.
4.1.3 Status Messages	AA	Supports	Save state announced via a role="status" live region.

SECTION 5

KNOWN LIMITATIONS

Three limitations are documented. Each has a mitigation, and none affects the operability of the application itself.

- **Colors inside user maps follow the historical MindMup theme.** Because renders existing academic .mup files exactly as MindMup drew them, so instructors' course materials look the way they were authored; a few of those inherited colors (for example the blue claim-number badges) fall short of 4.5:1. Mitigations: dark mode provides an alternative luminance scheme, and the structural information those colors carry is duplicated in shape, text labels, and the ARIA tree.
- **Connector lines are thin pointer targets.** The lines linking claims can be hard to click precisely. Every connector command is equally available from the keyboard-accessible menus, so the line is never the only way to act on a connection.

- **Toolbar hints use native tooltips.** Button hints rely on the browser's built-in title tooltip rather than a custom styled one. The same information is available to assistive technology through each button's accessible label.

SECTION 6

VERIFICATION AND MAINTENANCE

Accessibility in Because is a tested invariant, not a one-time audit. A dedicated regression suite runs before every deployment and must pass in full; it currently comprises 47 checks, including five axe-core scans (zero violations tolerated), end-to-end keyboard walkthroughs of the menus, dialogs, toolbar, and map editing commands in WebKit, and assertions against the accessibility tree Chrome computes — the representation Windows screen readers consume. The measures described in this report, the complete contrast palette, and the limitations above are maintained in the project's accessibility documentation alongside the source code, which is MIT-licensed for release as open source.

Questions or accommodation needs: contact Simon Cullen at sc@simoncullen.org. Accessibility issues are treated as functional defects: reproduced, fixed, and added to the regression gate so they cannot recur.